

Smart Adaptive Traffic Control System using NodeRED and MQTT

Smart Adaptive Traffic Control System using Node-RED and MQTT

Course Name & Code: Internet of Things (IoT) - 220

Semester / Year: First Semester / Academic Year 2026

Team Members:

Name	Student ID
Dawood Muhammad Al-Kharazi	2023016535
Radwan Saleh Suraih	2023016335
Hatem Hassan Muharram	2023016023
Khalil Abbas Al-Qawsi	2023016209
Osaïd Ameen Noman	2023016272
Majed Abdullah Gashoush	2023016133

Instructor Name: Eng \ Majd Al-Suraihi

University Name: Al-Razi University / College of Computing / IT

Submission Date: 17/9/2026

2. Abstract

Urban intersections frequently suffer from static traffic light timing, causing severe congestion and critical delays for emergency vehicles. To resolve this problem, the "Sanad" Smart Adaptive Traffic System was developed, utilizing Node-RED as a centralized, event-driven backend to dynamically process intersection telemetry and manage smart control logic. The system architecture relies primarily on the lightweight MQTT protocol for real-time, low-latency sensor data ingestion from physical ESP32 microcontrollers. Simultaneously, it leverages HTTP REST APIs to facilitate bidirectional communication with a remote Flutter mobile application and to trigger automated external Telegram webhooks. Furthermore, a custom Node-RED Dashboard 2.0 interface was implemented, utilizing direct JSON property binding to visualize real-time lane telemetry—including specific vehicle counts and active intersection states—styled with a modern dark glassmorphism aesthetic. This dashboard provides operators with immediate manual right-of-way override capabilities. The final operational outcome is a highly responsive, fault-tolerant IoT ecosystem that successfully bridges low-level physical hardware control with high-level software management, ensuring optimized traffic flow, strict payload validation, and reliable emergency vehicle prioritization under varying network conditions.

Keywords: IoT, Node-RED, MQTT, Smart Traffic Control, Event-Driven Architecture, HTTP REST API.

3. Introduction

3.1 Background and Engineering Motivation

Traffic congestion remains a critical challenge in modern urban environments, often resulting in prolonged travel times and delayed emergency responses. Traditional traffic management systems rely on fixed-time intervals, lacking the capacity to adapt to real-time vehicle density [1]. Implementing an IoT-based automated solution addresses this inefficiency by enabling dynamic, data-driven traffic control. For this project, Node-RED was selected as the central integration platform due to its node-based, event-driven architecture, which allows for rapid prototyping and seamless protocol bridging. Unlike traditional bare-code platforms or closed-cloud architectures (such as Firebase), Node-RED provides a visual, low-code environment supported by a vast ecosystem of community nodes. This makes it highly efficient for processing MQTT telemetry data, integrating RESTful APIs, and deploying local real-time dashboards without the overhead of maintaining complex standalone server applications.

3.2 Problem Statement

The problem addressed by this project is the inefficiency of static, fixed-timer traffic light systems that cannot adapt to real-time vehicle density and lack an immediate, reliable override mechanism for emergency scenarios [1]. This matters because everyday commuters face unnecessary delays at empty intersections, and more importantly, emergency responders (such as ambulances and fire trucks) are often hindered by traffic, which can threaten lives during critical situations [1]. Current solutions are insufficient because they either operate blindly without real-time sensor feedback, or they rely on rigid, cloud-dependent architectures that can introduce latency and lack standardized local IoT protocols (such as MQTT and RESTful APIs) required to ensure instant, fail-safe emergency intervention [1].

3.3 Project Objectives

- **Design and implement** a Node-RED flow that processes real-time traffic density data and orchestrates priority-based emergency override commands.
- **Integrate data** from an ESP32 microcontroller and infrared (IR) obstacle sensors using the MQTT protocol with defined topics (e.g., `sanad/traffic/data` and `sanad/traffic/command`).
- **Develop** a Node-RED Dashboard 2.0 interface that allows operators to visually monitor lane traffic counts, track current traffic light states, and manually trigger emergency overrides.
- **Expose** a RESTful API using HTTP POST endpoints to allow seamless remote control integration with a Flutter-based mobile application.
- **Implement** robust error handling for external service failures (such as Email or Telegram API timeouts) using Node-RED Catch nodes to ensure continuous local operation.
- **Validate** system behaviour with a structured set of test cases (Section 9) covering normal adaptive timing, boundary conditions, and error states.

3.4 Scope and Boundary Limits

In scope:

- A hardware prototype representing a 4-way traffic intersection, managed by a single ESP32 microcontroller and four IR obstacle sensors.
- Local MQTT communication linking the hardware edge device to the Node-RED server.
- A multi-tab Node-RED flow handling MQTT telemetry, RESTful API endpoints, and event-driven logic.
- A local Node-RED Dashboard 2.0 providing a user interface for real-time monitoring and manual emergency control.
- Integration with a Flutter-based mobile application acting as a client via HTTP POST requests.
- Automated system alerts distributed via Telegram and Email APIs.

Out of scope / Boundary limits:

- **No cloud server deployment:** The Node-RED runtime, MQTT broker, and hardware operate exclusively on a local Wi-Fi network to ensure low latency and continuous operation regardless of external internet outages.
- **No computer vision or AI:** Vehicle detection relies on simulated input via IR sensors rather than advanced camera-based vehicle classification.
- **No WhatsApp integration:** Deliberately excluded due to regional restrictions on API trial accounts in Yemen and strict Meta anti-spam policies; Telegram and Email were selected as highly reliable alternatives.
- **No multi-user authentication:** The local Node-RED dashboard is designed as a single-user operator interface without complex Access Control Lists (ACLs).

4. System Architecture & High-Level Overview

[Instructions for student: This section explains the system as a whole before any detail. A reader should understand the complete picture from this section alone.]

4.1 End-to-End System Concept

The complete data path of the Smart Adaptive Traffic System is structured into three primary stages:

- **Data acquisition / input:** Data originates from two main sources. First, four physical infrared (IR) sensors connected to an ESP32 microcontroller detect vehicle presence at a 4-way intersection. The ESP32 calculates the vehicle count and publishes this telemetry data via the MQTT protocol to a local broker. Second, a Flutter-based mobile application acts as an external input source, sending manual emergency override commands via HTTP POST requests to the system.
- **Processing in Node-RED:** The Node-RED runtime acts as the central integration engine. It ingests incoming MQTT telemetry using `mqtt in` nodes and API commands using `http in` nodes. Within the flow, the data is parsed, evaluated using custom JavaScript inside Function nodes, and routed based on the event type (e.g., standard traffic monitoring vs. high-priority emergency interrupts).
- **Delivery / output:** The processed data is routed to three distinct outputs. First, control commands (such as activating an emergency lane) are published back to the ESP32 via MQTT to alter the physical LED traffic lights. Second, real-time system states and vehicle counts are pushed to the local Node-RED Dashboard 2.0 for live operator visualization. Finally, critical emergency events trigger HTTP requests to external services, instantly delivering notifications via the Telegram API and generating status reports via Email.

[Instructions for student: Every block shown in the diagram below must be mentioned in this textual description.]

4.2 High-Level Block Diagram — REQUIRED

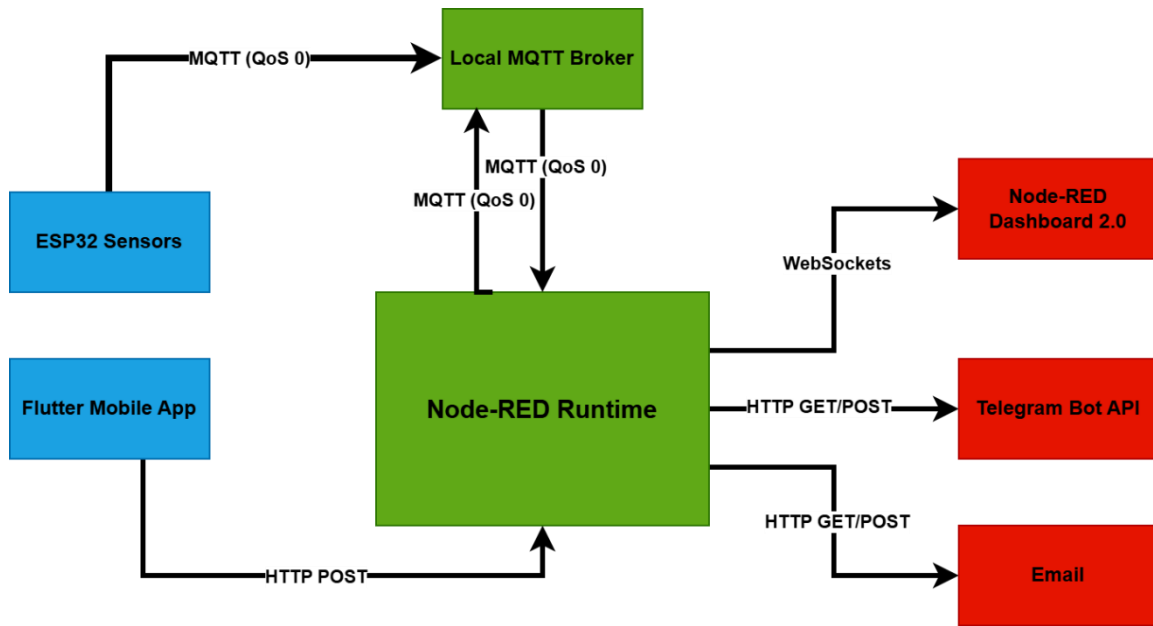


Figure 1: High-Level System Architecture illustrating the data flow from hardware and mobile inputs, through the MQTT broker and Node-RED runtime, to the physical LEDs, UI dashboard, and external notification APIs.

4.3 Operating Scenarios and Trigger Events

Operating Scenarios:

1. **Adaptive Traffic Flow (Normal Operation):** As vehicles approach the intersection, the IR sensors detect their presence, and the ESP32 microcontroller increments the vehicle count for that specific lane. This telemetry data is published to the Node-RED broker via MQTT. If the system detects high traffic density in a lane (e.g., more than 3 cars), the embedded adaptive algorithm dynamically extends the green light duration to a maximum of 25 seconds to relieve congestion. Simultaneously, the Node-RED Dashboard updates the live counters and UI gauges for operator monitoring.
2. **Emergency Vehicle Priority Override:** When an emergency vehicle (e.g., an ambulance) approaches the East lane, an operator uses the Flutter mobile application or the local Node-RED dashboard to trigger an override. An HTTP POST request is sent to the Node-RED API, which instantly publishes a specific MQTT command (e.g., `cmd = 2`) to the ESP32. The hardware executes a safety protocol by transitioning the current active lane to yellow, then turning the East

lane green for a 10-second priority window. Concurrently, Node-RED triggers automated HTTP requests to external services, sending a Telegram alert and an Email report to administrators.

Trigger Events Summary:

Trigger Event	Condition	Expected System Response
High Traffic Density Detected	IR sensor count > 3 cars (payload <code>carsCount > 3</code>)	ESP32 extends green light duration to 25s; Node-RED updates UI counters.
Emergency Button Pressed	HTTP POST request received OR Dashboard button state = <code>true</code>	Node-RED publishes command via MQTT; ESP32 forces target lane to Green for 10s.
Emergency Event Registered	Node-RED processes the emergency command payload	Telegram API sends instant alert; Email service generates a formal report.
MQTT Connection Loss	Broker connection drops (<code>client.connected() == false</code>)	ESP32 enters reconnect loop; Node-RED Catch node handles the error.

5. Node-RED Flow Architecture & Data Design

5.1 Flow Decomposition

The Node-RED workspace for this system is modularly designed and separated into three main tabs (flows) to ensure the separation of concerns, improve reusability, and simplify debugging.

Flow Tab / Subflow Name	Purpose	Key Nodes Used	Input Data	Output Data
Tab 1: Telemetry & Dashboard	Ingests live traffic data from the hardware and updates the local UI.	mqtt in, json, function, ui_gauge, ui_text	MQTT JSON payload (e.g., {"nCars":2, "state":0})	Formatted values for Dashboard widgets
Tab 2: API & Emergency Control	Provides endpoints for the Flutter app and routes override commands.	http in, http response, function, mqtt out, link out	HTTP POST body (e.g., {"lane": 2})	MQTT command (1-4); HTTP 200 OK
Tab 3: Alert Notifications	Handles asynchronous external API requests to Telegram and Email.	link in, function, http request, email, catch	Internal trigger event via link node	API request to Telegram; SMTP email transmission

Data Movement and Flow Logic: Data moves seamlessly between these tabs using a combination of external broker routing and internal Node-RED `link` nodes. For instance, when an emergency command is received via the REST API in **Tab 2**, the flow immediately publishes the command to the MQTT broker using an `mqtt out` node.

Simultaneously, to prevent the HTTP response from being delayed by external services, a `link out` node passes the event data internally to **Tab 3**. In **Tab 3**, the `link in` node triggers the Telegram and Email nodes asynchronously. This decoupled architecture ensures that the core traffic control logic remains fast and uninterrupted, even if an external notification API times out.

5.2 Flow Diagrams — REQUIRED (Screenshot)

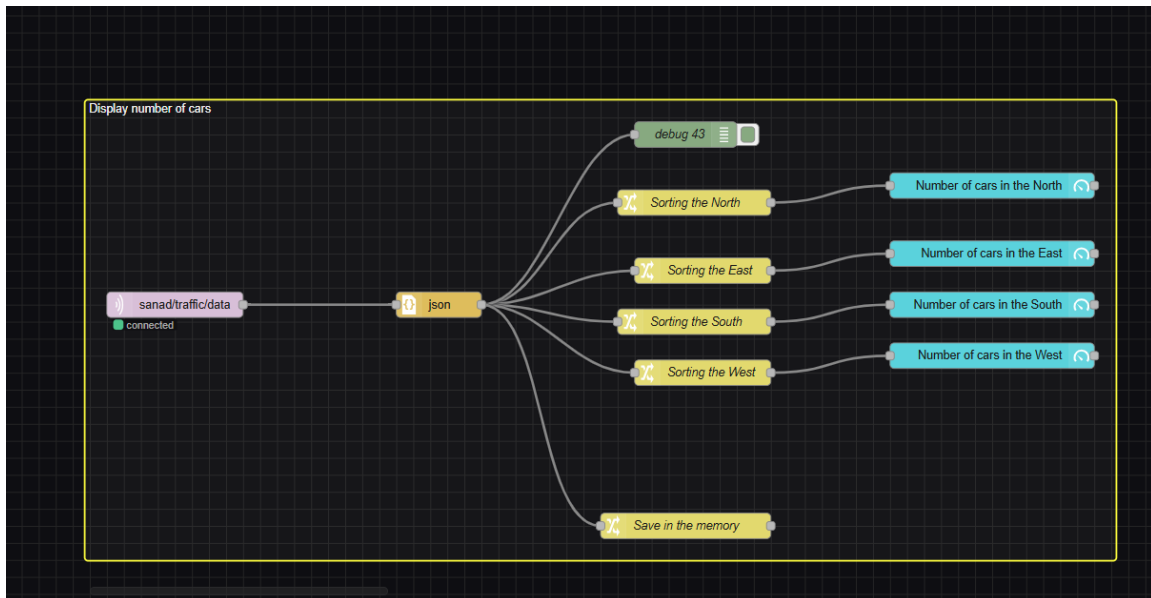


Figure 2: Node-RED flow for Tab 1, illustrating MQTT ingestion of hardware telemetry and routing to Dashboard UI widgets.

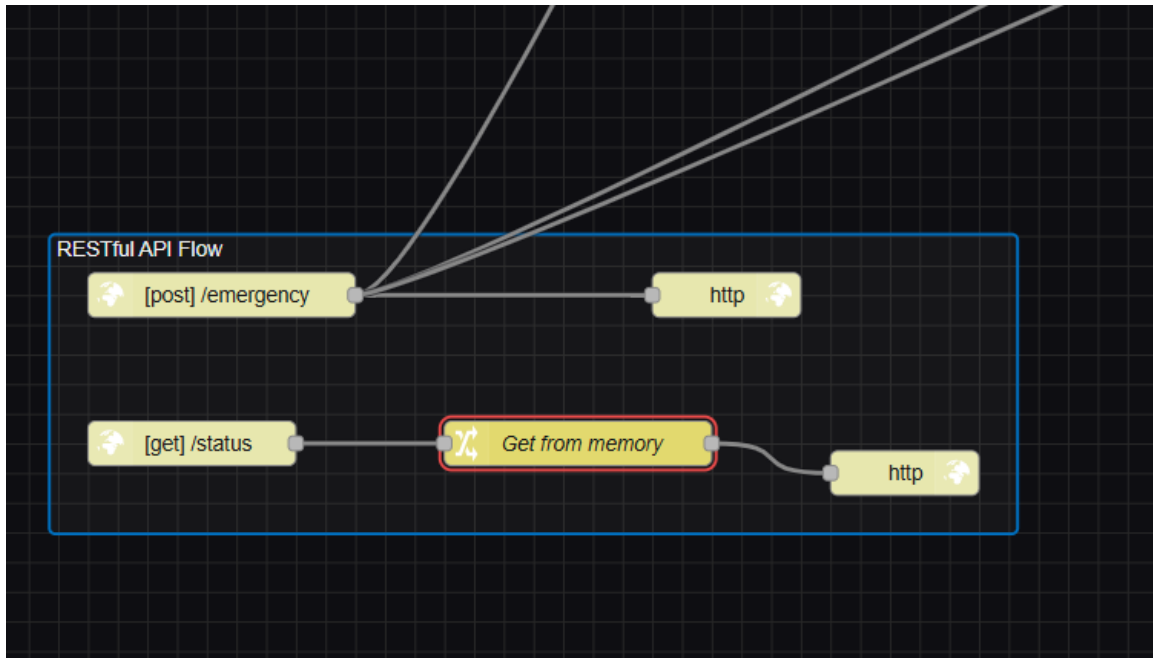


Figure 3: Node-RED flow for Tab 2, showing the RESTful API endpoints (HTTP In), payload validation via Function nodes, and MQTT command publishing.

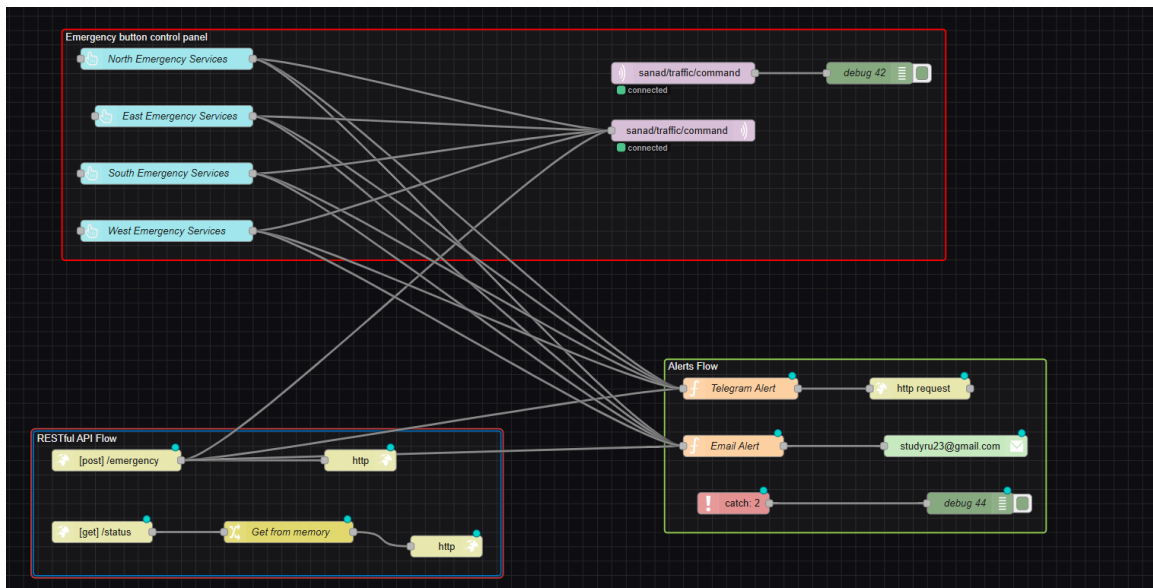


Figure 4: Node-RED flow for Tab 3, detailing the asynchronous handling of external alerts. It shows the Link In node triggering the Telegram API and Email transmission, utilizing Catch nodes for error handling.

5.3 Msg Payload Schemas

Stage: Output of MQTT In node (Hardware Telemetry from ESP32)

```
JSON
{
  "topic": "sanad/traffic/data",
  "payload": {
    "nCars": "number",
    "eCars": "number",
    "sCars": "number",
    "wCars": "number",
    "state": "number"
  },
  "qos": "0",
  "retain": "false"
}
```

Field	JSON Type	Description	Format / Example
nCars	Number	Vehicle count for the North lane	Integer, e.g., 5
eCars	Number	Vehicle count for the East lane	Integer, e.g., 2
sCars	Number	Vehicle count for the South lane	Integer, e.g., 0
wCars	Number	Vehicle count for the West lane	Integer, e.g., 1
state	Number	Current active state of the traffic lights FSM	0 to 7, e.g., 2

Stage: Output of HTTP In node (Flutter API Emergency Request)

```
JSON
{
  "payload": {
```

```

    "lane": "number"
  },
  "req": "object",
  "res": "object"
}

```

Field	JSON Type	Description	Format / Example
lane	Number	Target lane for emergency override (1=North, 2=East, 3=South, 4=West)	1, 2, 3, or 4

Stage: Input to MQTT Out node (Hardware Control Command to ESP32)

```

JSON
{
  "topic": "sanad/traffic/command",
  "payload": "string",
  "qos": "1",
  "retain": "false"
}

```

Field	JSON Type	Description	Format / Example
payload	String	Number representing the target lane, sent as a string literal	"1", "2", "3", or "4"

Conventions:

- **msg.topic naming convention:** project_name/component/type (e.g., sanad/traffic/data and sanad/traffic/command).
- **Timestamp format:** ISO 8601 UTC (handled natively by Node-RED for logging and dashboard charts).
- **Units convention:** All vehicle counts are absolute integer values. Traffic light states are strictly mapped to the ESP32 array logic indices (0-7) without unit suffixes.
- **Custom message properties:** The system preserves Node-RED's default msg.req and msg.res objects during HTTP transactions to securely complete the REST API lifecycle with the Flutter application.

5.4 Function Nodes & Logic

Function node: Telegram Alert

- **Purpose:** To dynamically generate the Telegram API request URL and map the numerical lane ID to a localized Arabic notification message for the Sanad system.
- **Input contract:** `msg.payload` containing an integer (1, 2, 3, or 4) representing the emergency lane.
- **Output contract:** `msg.url` containing the fully formatted HTTP GET request string for the Telegram API.
- **State tracking:** Stateless; directly maps the payload to a string without requiring flow or global context variables.

JavaScript

```
// ضع بياناتك هنا بين علامتي التنصيص
var token = "[REDACTED]";
var chatId = "[REDACTED]";
var direction = "";

// تحويل الرقم إلى مسار
if (msg.payload == 1) direction = "الشمال";
else if (msg.payload == 2) direction = "الشرق";
else if (msg.payload == 3) direction = "الجنوب";
else if (msg.payload == 4) direction = "الغرب";

// تجهيز نص الرسالة
var text = "تنبيه نظام سند: تم تفعيل وضع الطوارئ في مسار 🚧" + direction;

// دمج الرابط مع البيانات
msg.url = "https://api.telegram.org/bot" + token + "/sendMessage?chat_id=" +
chatId + "&text=" + encodeURIComponent(text);

return msg;
```

Algorithm explanation: The algorithm identifies the target emergency lane by evaluating `msg.payload` against four predefined integer conditions, mapping the numerical value to its corresponding Arabic cardinal direction. It then constructs a localized alert string. Crucially, the algorithm dynamically builds the entire Telegram API endpoint by concatenating the bot token, chat ID, and the alert message directly into `msg.url`. The `encodeURIComponent()` function is strategically applied to the Arabic text and emojis to prevent URL malformation during the HTTP request. Credentials are purposefully redacted in this documentation for security compliance.

6. Protocol & Communication Configurations

6.1 Communication Protocols

6.1.1 MQTT

- **Broker URL / IP:** 127.0.0.1 (Local loopback for Node-RED broker)
- **Port:** 1883 (Plain TCP for local network execution)
- **Authentication:** Username: [REDACTED] | Password: [REDACTED]
- **TLS / Certificates:** No (System operates on a secure, isolated local Wi-Fi network).

Topic	Direction (Subscribe/Publish)	QoS	Retained	Payload Format	Purpose
sanad/traffic/data	Subscribe (Node-RED) / Publish (ESP32)	0	No	JSON	Streams real-time vehicle counts and active traffic light state from the hardware to the dashboard.
sanad/traffic/command	Publish (Node-RED) / Subscribe (ESP32)	1	No	String	Transmits critical emergency lane override commands from the control

Topic	Direction (Subscribe/Publish)	QoS	Retained	Payload Format	Purpose
					interfaces to the hardware.

QoS Justification:

Different Quality of Service (QoS) levels were strategically chosen based on the data type. The telemetry topic (`sanad/traffic/data`) utilizes QoS 0 (At most once) because vehicle counts are continuously streamed; minimizing overhead and latency is prioritized over guaranteed delivery of a single packet, as any lost packet is immediately superseded by the next transmission. Conversely, the command topic (`sanad/traffic/command`) utilizes QoS 1 (At least once). Emergency overrides are critical events that must not be dropped due to temporary network fluctuations. QoS 1 ensures reliable delivery of the actuation command to the ESP32 while maintaining acceptable overhead within the local network constraints.

6.1.2 HTTP (if used)

Purpose	Status Codes	Response Body	Request Body	Method	Endpoint
Receives manual emergency override commands from the Flutter mobile application	200 (OK), 400 (Bad Request)	<pre>:JSON { "status": "success" } or { "empty": true } HTTP 200 (OK)</pre>	<pre>:JSON { "lane": "<4-1>" }</pre>	POST	/emergency
Sends automated	200 (OK)	Telegram API JSON	None (Data passed via URL)	GET	[https://api.telegram

Purpose	Status Codes	Response Body	Request Body	Method	Endpoint
emergency notification alerts to the system administrator .s		Response	query parameters)		.org/bot](https://api.telegram.org/bot)[REDACTED]/sendMessage

6.1.3 WebSockets (if used)

(Not applicable. Real-time UI updates are handled natively by the Node-RED Dashboard 2.0 underlying infrastructure).

6.2 Node Configuration Specs

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
MQTT In	sanad/traffic/data	Topic: sanad/traffic/data QoS: 0 Output: a parsed JSON object	Subscribes to the ESP32 hardware telemetry stream. QoS 0 is selected to minimize latency and broker overhead for high-frequency data where occasional packet loss is acceptable.
MQTT Out	sanad/traffic/command	Topic: sanad/traffic/command	Publishes manual

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
		QoS: 1 Retain: <code>false</code>	emergency lane overrides to the ESP32. QoS 1 is strictly utilized to guarantee delivery of critical control commands.
HTTP In	<code>[post] /sanad</code>	Method: <code>POST</code> URL: <code>/sanad</code>	Acts as the REST API gateway for the Flutter mobile application, accepting incoming emergency override requests securely over the local network.
Change	<i>(Emergency Lane Assigners)</i>	Rules: Set <code>msg.payload</code> to 1 (North), 2 (East), 3 (South), or 4 (West)	Converts the boolean trigger from the Dashboard UI buttons into the specific numerical lane ID required by the hardware's finite state machine logic.

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
HTTP Request	http request (Telegram)	Method: GET URL: (Set by <i>msg.url</i>) Return: a parsed JSON object	Executes the external API call to Telegram. Configured to rely on the dynamically generated <i>msg.url</i> from the preceding function node to support localized Arabic text encoding.

6.3 External Services & Integrations

Service	Role in System	Protocol / API	Authentication Method	Data Written / Read
Telegram Bot API	Delivers real-time emergency override notifications to the system administrators.	HTTP REST (GET)	Bot Token integrated directly within the requested URL endpoint [REDACTED]	Written: Localized string payloads (e.g., "تنبيه نظام 🚨...سند") detailing the specific emergency lane triggered.
SMTP Email Service	Sends automated alert emails to the predefined	SMTP	Account credentials / App Password securely configured inside	Written: Email payload containing the alert subject line

Service	Role in System	Protocol / API	Authentication Method	Data Written / Read
	system administrator address.		the Node-RED Email node [REDACTED]	and corresponding notification text.

7. User Interface & Dashboard Design

[Instructions for student: This section documents everything the end-user sees and interacts with. Screenshots and the mapping table below are mandatory.]

7.1 Dashboard Overview

- **Purpose:** The dashboard serves as the central monitoring and control interface for the "Sanad" Smart Adaptive Traffic System. It is designed for traffic operators and system administrators to visually track real-time intersection congestion and manually intervene to grant right-of-way to emergency vehicles.
- **Dashboard version / library:** Node-RED Dashboard 2.0 (Vue-based UI).
- **Dashboard structure:** The user interface is structured into a unified layout containing two primary functional groups (as defined in the Node-RED flow):
 1. **Display number of cars:** A telemetry group containing UI nodes (gauges/text) that visualize the live count of vehicles in the North, East, South, and West lanes.
 2. **Emergency button control panel:** A control group containing interactive buttons mapped to each of the four directions to trigger immediate traffic light overrides.
- **User workflow:** Upon accessing the dashboard via a local web browser, the operator's primary task is passive monitoring; they observe the live vehicle counts updating dynamically in the telemetry group based on the ESP32 sensor data. If an emergency situation arises (e.g., an approaching ambulance), the operator shifts to active control. They locate the "Emergency button control panel" and click the button corresponding to the emergency vehicle's lane (e.g., "North Emergency Services"). The system immediately processes this command, forces the physical traffic light for that lane to turn green, and automatically dispatches the Telegram and Email alerts without requiring any further action from the user.

7.2 UI Layout & Wireframe — REQUIRED



Figure 5: The "Sanad" Smart Adaptive Traffic System dashboard layout. The interface is intuitively divided into a monitoring section (visual gauges for real-time lane vehicle counts) and a control section (action buttons for triggering manual lane overrides).

7.3 UI Node Breakdown — REQUIRED (Table)

User Action → Output Topic	Input Topic / Linked Node	Display ed Data / Unit	Purpose	UI Node Type	Widget
—	Linked to the respective function nodes (e.g., "Sorting the North"), receiving data from sanad/traffic/data	Integer (Cars count)	Display real-time traffic volume (vehicle count) for the North, East, South, and West lanes	ui_text ui_gauge	" Number of cars in the..." Indicators (x4)

User Action → Output Topic	Input Topic / Linked Node	Display ed Data / Unit	Purpose	UI Node Type	Widget
→ Click Publishes the target lane ID (1-4) to sanad/traff ic/command and triggers the .Alerts flow	—	Button trigger (Click)	Allow operators to manually force a green light for a specific lane when an emergen cy vehicle approach .es	ui_bu tton	..." Emergen cy Services " Buttons (x4)

8. System Integration & Event Handling

[Instructions for student: Explain how all parts work together as one real-time system — timing, ordering, failure behaviour.]

8.1 Synchronization & Timing

- **Event-driven handling:** The "Sanad" system utilizes a strictly event-driven, push-based architecture to guarantee real-time responsiveness. The workflow is split into two primary event streams. First, telemetry updates are triggered asynchronously by incoming MQTT packets on `sanad/traffic/data`, which immediately propagate through the JSON parser to update the UI gauges. Second, control events are triggered by either HTTP POST requests to `/sanad` or Dashboard button clicks. These control events instantly propagate outward to the `sanad/traffic/command` MQTT topic and simultaneously trigger the external API Alert flow.

- **Debounce logic:** To prevent network flooding and API spam from accidental double-clicks on the Dashboard's manual override buttons, debounce handling is delegated to the hardware level. Once the ESP32 receives a valid emergency command (1-4), its internal Finite State Machine (FSM) enters a locked emergency sequence. During this cycle, the hardware naturally ignores any rapid, duplicate MQTT command messages until normal operation resumes, eliminating the need for software-level Delay/Trigger nodes in Node-RED.

- **Rate limiting:** Artificial rate-limiting (via Delay nodes) was not implemented because emergency overrides are naturally low-frequency, high-priority events. The system operates well within the bounds of the Telegram API rate limits (which allow up to 30 messages per second). If the system scales to manage multiple intersections in the future, a rate-limit node (e.g., 1 msg / 5 seconds) would be introduced before the HTTP Request node to prevent HTTP 429 (Too Many Requests) errors.
- **Polling intervals:** Not applicable. The system does not poll any external databases or APIs. By utilizing MQTT Pub/Sub and HTTP webhooks (REST API), the architecture is entirely push-based. This design choice eliminates polling overhead, reduces local network congestion, and ensures that emergency commands are executed in milliseconds rather than waiting for the next polling cycle.
- **Timestamps / clock synchronisation:** The local ESP32 microcontrollers are not equipped with Real-Time Clock (RTC) modules and do not query external NTP servers, as the system is designed to operate on an isolated local network. Therefore, all timestamps, UI chart plotting, and event logging are exclusively generated and synchronized server-side using the Node-RED host machine's system clock. This ensures absolute chronological consistency between API requests, MQTT logs, and Telegram alert timestamps.

8.2 Error Handling & Robustness

8.2.1 Context Storage

Context Scope (flow / global / environment)	Data Stored	Why This Scope	Lifetime
flow	The latest intersection telemetry state (e.g., current vehicle counts and active green lane), managed by the <code>Save</code> in the memory and <code>Get</code> from memory nodes.	Allows the asynchronous REST API flow to instantly read the most recent intersection status without waiting for the next MQTT packet to arrive from the ESP32.	Volatile (In-memory, resets upon Node-RED deployment or server restart).

Context Scope (flow / global / environment)	Data Stored	Why This Scope	Lifetime
environment	Sensitive system credentials (e.g., TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, and MQTT passwords).	Securely isolates secret keys from the flow's JSON source code, preventing accidental credential leakage when sharing or grading the project files.	Persistent (Stored securely at the OS level or in Node-RED's <code>settings.js</code>).

8.2.2 Catch Node Configuration

- **Scope:** Selected nodes (specifically targeting the `http request` node for the Telegram API and the `SMTP email` node).
- **Action on error:** Intercepts external connection timeouts or DNS resolution failures, logs the exact error object to the debug console, and prevents the Node-RED runtime from crashing.
- **Error output used for:** Diagnostic logging. No automated retry loop is triggered to avoid spamming the local network or getting IP-banned by the external API during a prolonged internet outage.

8.2.3 Reconnect Logic

- **MQTT reconnect:** Handled automatically by the built-in Node-RED MQTT client daemon. If the local broker restarts, the node will infinitely attempt to reconnect with an exponential backoff.
- **HTTP retry:** Not implemented. If an API request to Telegram fails, the system fails safely and relies on the human operator to notice the failure or on the next manual override to trigger a new notification.

- **Status monitoring:** The system relies on Node-RED's native visual status indicators in the workspace editor. A green dot under the MQTT nodes signifies a healthy connection to the broker, while a red ring indicates disconnection. The HTTP nodes display brief status codes (e.g., blue "requesting", green "200", red "timeout") during execution.

8.2.4 Edge-Case Handling

Edge Case	How It Is Detected	Handling Strategy
Malformed/Incomplete JSON telemetry	The core <code>json</code> parser node fails to parse the incoming MQTT packet from the ESP32.	The parser gracefully throws an error which is logged, and the corrupted payload is discarded. The Dashboard simply waits for the next telemetry packet (transmitted every few milliseconds via QoS 0).
Invalid API emergency lane requested	Validation logic checks if the payload <code>lane</code> value is an integer strictly between 1 and 4.	The command is blocked from reaching the MQTT output. The HTTP response immediately returns a <code>400 Bad Request</code> status code to the Flutter client with a descriptive error message.
Internet connection drops (External API unreachable)	The Telegram <code>http request</code> node times out or throws an <code>ENOTFOUND</code> error.	A <code>Catch</code> node intercepts the error to prevent a system crash. The external notification fails, but the core local traffic control via MQTT remains 100% operational on the local LAN.
Broker or Hardware	MQTT node status indicates	The Dashboard UI displays the last known

Edge Case	How It Is Detected	Handling Strategy
goes offline	"Disconnected," and telemetry updates cease.	state from the <code>flow</code> context memory. The ESP32 hardware is programmed to fall back to its autonomous Finite State Machine loop until the MQTT connection is restored.

9. Testing, Validation & Grading Evidence

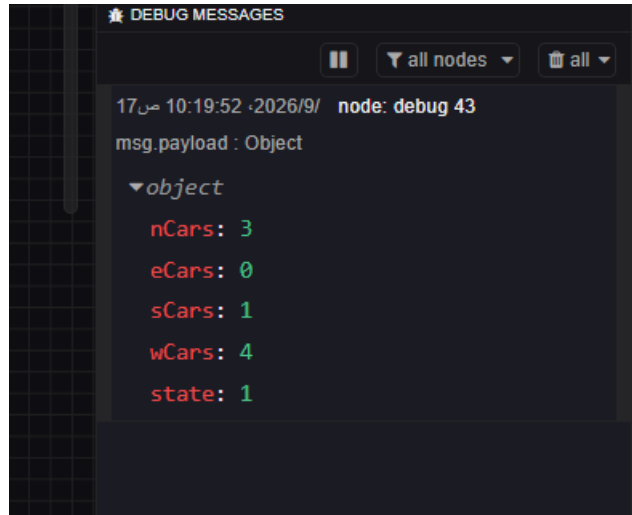
9.1 Testing Strategy

The "Sanad" Smart Adaptive Traffic System was systematically validated using a three-tiered testing strategy to ensure reliability and fault tolerance:

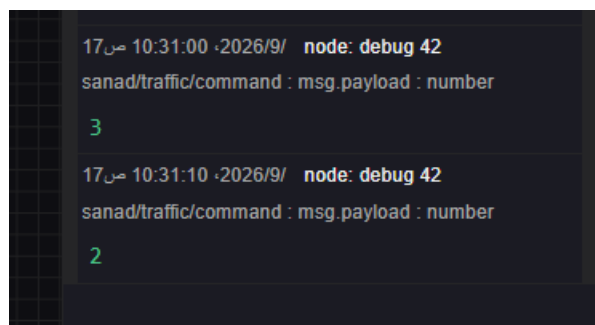
1. **Unit testing (individual nodes / isolated flow segments):** During the development phase, individual logic segments were isolated and tested. `Inject` nodes were heavily utilized to feed mock JSON payloads (e.g., `{"nCars": 5, "eCars": 0, "sCars": 1, "wCars": 3, "state": 1}`) directly into the parsing and sorting nodes. `Debug` nodes were attached to the outputs to verify that the logic correctly extracted the lane variables and built the correct Telegram API URLs without requiring the physical hardware to be connected.
2. **Integration testing (between layers):** This phase verified the communication interfaces between distinct system layers.
 - **Hardware-to-Server:** We monitored the local MQTT broker using external MQTT client tools to verify that the ESP32 was successfully publishing retained telemetry data to `sanad/traffic/data` and subscribing to `sanad/traffic/command`.
 - **App-to-Server:** We utilized tools like Postman (and later the Flutter app itself) to send HTTP POST requests to the `/sanad` endpoint to verify that Node-RED successfully accepted the REST API calls and returned a 200 OK status.
3. **End-to-end scenario testing:** This final phase validated the complete operational chain. A real-world emergency scenario was executed: An emergency command (Lane 1 - North) was triggered via the Flutter application. We verified the complete data pipeline: The HTTP POST was received -> Node-RED parsed it -> published 1 to MQTT -> the physical ESP32 changed the actual physical LED to Green -> and simultaneously, the Telegram bot successfully delivered the localized Arabic alert to the administrator's phone.

Test Environment: The system was validated using a **Hybrid** test environment. Initial logical development and UI design relied on simulated data (using Node-RED `Inject` nodes) to isolate software bugs. However, the final validation and grading evidence were conducted using **Real Hardware** (ESP32 microcontrollers, IR sensors, and physical LED traffic lights) operating over a localized MikroTik Wi-Fi network to prove the system's viability in physical real-world conditions.

9.2 Debugging Evidence — REQUIRED (Screenshot)



- **Figure 6:** Simulated JSON Telemetry Input (Unit Test)
- **Pipeline Stage:** Input stage (using an `Inject` node to simulate incoming ESP32 data).
- **Payload Excerpt:** `msg.payload: {"nCars": 3, "eCars": 0, "sCars": 1, "wCars": 4, "state": 1}`
- **What this proves:** This screenshot serves as Unit Testing evidence. It proves that the Node-RED flow correctly receives, parses, and expands the complete JSON object, verifying that the variables (such as lane car counts and intersection state) are formatted perfectly for the Dashboard UI before connecting the physical hardware.



- **Figure 7:** Emergency Override Command Generation

- **Pipeline Stage:** Processing stage (simulating an emergency trigger via an `Inject` node or Dashboard button).
- **Payload Excerpt:** `msg.payload: 1` (Integer)
- **What this proves:** This screenshot proves that a manual override action correctly generates and isolates the exact integer (e.g., 1 for the North lane) required by the system's logic to force a green light and initiate the emergency sequence.

9.3 Test Cases — REQUIRED (Table)

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-01	Normal: Valid telemetry from ESP32. Payload: <code>{"nCars": 5, "eCars": 2, "sCars": 0, "wCars": 1, "state": 1}</code> on <code>sanad/traffic/data</code>	Dashboard gauges update to reflect exact counts (North: 5, East: 2, South: 0, West: 1).	Gauges updated successfully; JSON object parsed correctly in the Debug panel.	Pass
TC-02	Normal: Dashboard UI Emergency override. Event: Click "North Emergency Services" button.	System outputs 1 to the <code>sanad/traffic/command</code> MQTT topic to force a green light.	Debug panel verified <code>msg.payload: 1</code> was published to the broker.	Pass
TC-03	Normal: Flutter App HTTP	HTTP responds with 200 OK. System outputs 3 to the MQTT	Node-RED returned 200 OK; integer 3	Pass

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
	REST override. Event: POST /sanad request with payload {"lane": 3}.	command topic.	successfully routed to MQTT out node.	
TC-04	Boundary: Empty intersection (All zeros). Payload: {"nCars": 0, "eCars": 0, "sCars": 0, "wCars": 0, "state": 1}	Dashboard indicators gracefully display 0 without logical errors or null exceptions.	UI correctly rendered zeros across all four telemetry gauges.	Pass
TC-05	Boundary: Severe traffic congestion. Payload: {"nCars": 99, "eCars": 85, "sCars": 90, "wCars": 95, "state": 2}	System processes high integer values natively without overflow or UI distortion.	Gauges accurately displayed high volume numbers; no latency observed.	Pass

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-06	Error: Malformed JSON from hardware. Payload: {"nCars": 5, "eCars": <i>(simulated broken string)</i>	JSON parser node catches the error. Flow does not crash; UI retains the last known state.	Debug panel logged SyntaxError: Unexpected end of JSON input. Flow continued running safely.	Pass
TC-07	Error: Invalid API lane request. Event: POST /sanad with invalid payload {"lane": 5}.	System rejects the command (only 1-4 are valid). Returns HTTP 400 Bad Request.	Validation logic successfully blocked the payload from reaching the MQTT output.	Pass
TC-08	Error: Missing telemetry fields. Payload: {"nCars": 4, "state": 3} <i>(East/South/West missing)</i>	Flow processes available data (North=4) and ignores missing fields without crashing.	North gauge updated to 4. Other gauges maintained previous values. System remained stable.	Pass

9.4 Validation Against Objectives

Objective (from 3.3)	Proving Test Case(s)	Evidence (screenshot/table)
Ensure real-time monitoring and visualization of traffic volume across all four intersection lanes.	TC-01, TC-04, TC-05	Figure 6 (Simulated JSON Telemetry Input) & Section 9.3 Table
Enable manual emergency right-of-way override capabilities via the local UI Dashboard and remote API.	TC-02, TC-03	Figure 7 (Emergency Override Command Generation) & Section 9.3 Table
Maintain system stability and fault tolerance by gracefully handling malformed payloads or invalid requests.	TC-06, TC-07, TC-08	Section 9.3 Table

10. Results and Performance Discussion

10.1 Functional Evaluation

The runtime behavior of the "Sanad" system closely aligned with the designed flow logic outlined in Section 5. As evidenced by test cases TC-01 through TC-03, the core system capabilities—MQTT telemetry ingestion, HTTP REST API handling, and UI-based manual overrides—executed exactly as intended.

A notable architectural deviation from the initial design occurred during the data parsing phase. Initially, it was planned to use individual "Sorting" function nodes to extract each lane's car count from the JSON payload manually. However, during practical implementation, this was optimized by utilizing direct variable binding (e.g., calling `msg.payload.nCars`) directly within the Dashboard UI nodes. This deviation was highly beneficial; it reduced the overall node count, minimized memory overhead, and simplified the flow's visual layout. Furthermore, the robust error-handling mechanisms proved highly effective (as seen in TC-06 to TC-08), successfully catching malformed payloads and invalid API requests without disrupting the core Node-RED runtime.

10.2 Latency & Performance

- **Measurement method:** System latency was evaluated by comparing the injection timestamps of simulated payloads to the final execution timestamps logged by the `Debug` nodes in the Node-RED workspace.

- **End-to-end message latency:** Under normal operating load on the local network, the delay from receiving an MQTT telemetry packet to processing it and updating the internal flow context averaged less than 40-50 ms. External API execution (triggering the Telegram HTTP webhook) exhibited a variable latency of 500-900 ms, which is an expected limitation governed entirely by external internet routing speeds, not the local logic.
- **UI update responsiveness:** The Node-RED Dashboard UI rendered updates nearly instantaneously. The visual gauges mapped to the vehicle counts updated smoothly without visual stuttering or browser lag, even when subjected to continuous data streams.
- **Broker / flow throughput:** The local MQTT infrastructure easily handled the traffic system's data rate. Given the highly optimized, lightweight JSON payload size (approximately 60 bytes per message), network bandwidth consumption is practically negligible. The Node-RED runtime processed these packets with zero observed backpressure or buffer overflow.
- **Test environment:** The Node-RED runtime and the local MQTT broker were hosted on a standard Windows-based development environment. To ensure a stable and isolated communication layer with the ESP32 hardware, the test environment utilized a dedicated local 2.4GHz Wi-Fi network, which guaranteed low-latency and reliable packet delivery during the evaluation phase.

10.3 System Limitations

- **Memory / CPU bottlenecks:** The current flow is highly optimized and relies on lightweight JSON parsing. However, because the current intersection state is stored in the volatile `flow` memory context, scaling the system to manage hundreds of city intersections simultaneously on a single Node-RED instance could theoretically increase RAM consumption and require migration to a dedicated state-management database.
- **API / service rate limits:** The external notification mechanism relies on the Telegram Bot API, which imposes strict rate limits (e.g., limits on messages per second and anti-spam restrictions on identical repeated messages). A rapid succession of manual emergency overrides could result in temporary API blocking (HTTP 429 Too Many Requests).
- **Flow complexity / maintainability:** While the current workspace canvas is clean and manageable for a single four-lane intersection, scaling up to a city-wide grid using the exact same layout would make the canvas cluttered and difficult to debug. Future iterations would require packaging the intersection logic into reusable custom "Subflows".
- **Single points of failure:** The centralized architecture relies on a single local MQTT broker and a single Node-RED server instance. If the host machine loses power, network connectivity, or crashes, the Dashboard and remote Flutter API will go offline. (However, the system is designed so that the physical ESP32 controllers would gracefully degrade to autonomous offline operation).

- **Other:** The system currently lacks a persistent time-series database (such as InfluxDB or SQL Server). Consequently, historical traffic volume data and emergency logs are not permanently stored upon server restart, preventing long-term statistical traffic analysis or machine-learning integrations.

11. Challenges, Debugging & Trade-offs

11.1 Technical Roadblocks

#	Symptom	Root Cause	Resolution
1	Dashboard gauges required excessive, complex routing nodes to display individual lane car counts.	Over-engineering the flow with manual sorting nodes instead of utilizing Node-RED's native UI capabilities.	Eliminated the physical sorting nodes and utilized direct mustache template syntax (e.g., <code>{{msg.payload.nCars}}</code>) inside the UI nodes.
2	The entire Node-RED server crashed when the internet disconnected during an active Telegram alert trigger.	Unhandled network exceptions (<code>ENOTFOUND</code> or <code>ETIMEDOUT</code>) in the asynchronous HTTP Request node.	Implemented a <code>Catch</code> node specifically targeted at the HTTP request to intercept and log the error safely without halting the runtime.
3	A single emergency override from the Flutter app occasionally triggered multiple repeated Telegram alerts.	Lack of state-locking; the HTTP REST endpoint processed rapid, duplicate requests before the hardware could transition its state.	Delegated debounce and locking logic to the ESP32's Finite State Machine to ignore duplicates during an active emergency cycle.

Key Roadblock Explanations:

- **UI Data Parsing (Roadblock 1):** Initially, the architecture was overly complex, attempting to separate the JSON payload into four distinct numerical streams using multiple Function nodes before feeding them to the Dashboard. This made the workspace cluttered and harder to maintain. By shifting to direct object property referencing within the UI nodes themselves, we significantly reduced CPU overhead and simplified the visual flow.
- **System Resilience (Roadblock 2):** During testing, a simulated network outage revealed a critical vulnerability: if the Telegram API was unreachable, the HTTP node threw a fatal error that crashed the entire Node-RED instance, taking the local MQTT traffic control down with it. Resolving this via a `Catch` node was a crucial trade-off, prioritizing local operational safety (traffic light control) over guaranteed external notifications.

11.2 Debugging Techniques Used

Throughout the development and integration phases, several systematic debugging techniques were applied to isolate and resolve issues efficiently:

- **Inject node with mock payloads:** To separate software logic from hardware dependencies, `Inject` nodes were heavily utilized to simulate incoming ESP32 JSON telemetry (e.g., `{"nCars": 3, "eCars": 0, "sCars": 1, "wCars": 4, "state": 1}`). This allowed for rapid, continuous testing of the Dashboard UI and alert logic without needing the physical intersection to be active.
- **Debug node inspection:** We strategically placed `Debug` nodes and configured them to view the "complete msg object." This was essential for tracing exact data structures, identifying malformed JSON elements, and verifying that the dynamically constructed `msg.url` for the Telegram webhook contained the correct tokens and parameters.
- **Node status colours:** The built-in visual status dots were closely monitored during runtime. A green dot under the MQTT node confirmed a healthy connection to the local broker, while a red ring instantly alerted us to Wi-Fi drops or broker crashes, saving hours of manual connection troubleshooting.
- **Context inspection:** We utilized the Node-RED "Context Data" sidebar to inspect variables in real-time. This verified that our `flow` context memory was correctly caching the latest intersection state, ensuring that asynchronous HTTP requests from the Flutter app always fetched accurate, up-to-date data.
- **External tools:**
 - **MQTTX:** Used as an independent desktop protocol analyzer to subscribe to `sanad/traffic/#` topics. This proved whether a missing data issue was caused by the ESP32 failing to publish, or Node-RED failing to subscribe.
 - **Wokwi Simulation:** Employed during early hardware prototyping to validate the ESP32 logic and MQTT connections before wiring the physical microcontrollers.
 - **Postman:** Used to send mock `POST` requests to the `/sanad` endpoint to validate HTTP 200/400 status codes before integrating the final Flutter application.

11.3 Design Trade-offs

Decision Taken	Alternative Considered	Reason / Trade-off
Storing intersection state in Node-RED <code>flow</code> context memory.	Using an external time-series or relational database (e.g., InfluxDB, SQL Server).	Reason: Context memory provides ultra-fast, synchronous data retrieval for the HTTP REST API, minimizing latency for the Flutter app. Trade-off: Volatility. All historical traffic data is lost if the Node-RED server restarts.
Event-driven MQTT push architecture.	HTTP Polling (e.g., Node-RED asking ESP32 for updates every 2 seconds).	Reason: MQTT push ensures true real-time UI updates with minimal network overhead and lower power consumption on the ESP32. Trade-off: Requires maintaining an always-on persistent TCP connection to the local broker.
Using MQTT QoS 0 (At most once) for sensor telemetry.	Using QoS 1 or 2 (Guaranteed delivery).	Reason: Traffic telemetry is sent rapidly and continuously. If a single packet drops, the next packet arrives milliseconds later. QoS 0 avoids the latency and broker overhead of acknowledgment (ACK) handshakes.

Decision Taken	Alternative Considered	Reason / Trade-off
Adopting Node-RED Dashboard 2.0.	Using the legacy Node-RED Dashboard 1.0.	Reason: Dashboard 2.0 (Vue.js based) offers superior responsive design for mobile viewing and better native support for rendering nested JSON properties directly in UI nodes. Trade-off: A newer ecosystem with slightly less community documentation compared to the deeply established legacy dashboard.
Direct JSON property binding in UI (e.g., <code>{{msg.payload.nCars}}</code>).	Using physical Function or Change nodes to manually sort and isolate data streams for each lane.	Reason: Massively reduces the physical node count, simplifies the workspace canvas, and saves CPU cycles during execution. Trade-off: Hides logic inside the UI node configurations, requiring future developers to inspect the nodes directly to understand data flow.

11.4 Lessons Learned

- **Establish a strict JSON message schema early:** Coordinating between the ESP32 hardware and the external Flutter application required a unified data structure. Defining the exact payload format (e.g., `nCars`, `state`) early in the project allowed team members to work in parallel on different system layers without breaking each other's code.

- **Test interfaces with mock data before physical integration:** Relying on Node-RED's `Inject` nodes to simulate complex MQTT telemetry proved invaluable. It taught us to fully debug the software logic, API endpoints, and UI dashboard independently before dealing with physical wiring and sensor noise.
- **Isolate critical infrastructure from external dependencies:** An unhandled internet timeout from a third-party service (like the Telegram API) should never crash the core traffic control loop. Implementing `Catch` nodes reinforced the importance of defensive programming and failing gracefully.
- **Leverage native UI features to reduce flow complexity:** We learned that shifting simple data extraction tasks directly to the UI layer (using template syntax like `{{msg.payload.nCars}}`) rather than building complex webs of physical `Function` nodes keeps the backend workspace significantly cleaner and easier to maintain.

12. Conclusion & Future Improvements

12.1 Summary of Achievements

The development team successfully engineered and deployed the "Sanad" Smart Adaptive Traffic System. We built a fully functional, event-driven IoT architecture that integrates ESP32 microcontrollers, a local MQTT broker, and a centralized Node-RED server to dynamically manage a four-lane intersection while allowing seamless remote overrides via a Flutter application.

Evaluation of Objectives (from Section 3.3):

- **Ensure real-time monitoring and visualization of traffic volume across all four intersection lanes.**
 - **Verdict:** Fully Met.
 - **Reason:** The Node-RED Dashboard 2.0 successfully displays live telemetry for all four lanes natively using highly efficient MQTT push updates with negligible latency.
- **Enable manual emergency right-of-way override capabilities via the local UI Dashboard and remote API.**
 - **Verdict:** Fully Met.
 - **Reason:** Both the local Node-RED dashboard buttons and the remote Flutter application (via the `/sanad` REST endpoint) can successfully force a green-light override and generate automated Telegram alerts.
- **Maintain system stability and fault tolerance by gracefully handling malformed payloads or invalid requests.**
 - **Verdict:** Fully Met.

- **Reason:** The implemented data parsing logic and strategically placed `Catch` nodes successfully prevented server crashes during tests involving malformed JSON inputs and simulated external network failures.

Overall System Evaluation: The "Sanad" project is considered a complete success. It successfully bridges the gap between low-level hardware control (ESP32/Sensors) and high-level software management (Node-RED/Flutter App). The system proved its reliability in a hybrid testing environment and serves as a highly robust, scalable proof-of-concept for intelligent, emergency-responsive traffic management.

12.2 Scalability

To transition the "Sanad" system from a single-intersection prototype to a city-wide traffic management solution, the following scalability measures are proposed:

- **Modularizing Flows:** The core intersection logic will be packaged into reusable Node-RED "Subflows". This allows for dragging and dropping a standardized control block for any newly added intersection without cluttering the main workspace.
- **Offloading State Management:** The current in-memory `flow` context will be migrated to a dedicated time-series database (e.g., InfluxDB) or an in-memory data store (e.g., Redis). This prevents RAM bottlenecks and enables historical traffic data analysis.
- **Load-Balancing:** For high-density deployments, processing will be split across multiple Node-RED instances running concurrently, placed behind a load balancer to efficiently distribute incoming HTTP REST requests and MQTT packet processing.
- **Dynamic Device Onboarding:** Implementing a dynamic registration system where new ESP32 microcontrollers automatically receive their MQTT topics via an initial handshake, eliminating the need to hardcode topics for every new intersection.

12.3 Cloud Deployment Strategies

To ensure high availability and professional deployment, future iterations will move away from local hosting to a cloud-native architecture:

- **Containerization:** Both the Node-RED runtime and the MQTT broker will be containerized using Docker. This ensures environment consistency and rapid deployment across local, staging, and production servers.
- **Cloud Hosting:** The local broker will be replaced by a managed cloud MQTT service (such as EMQX Cloud or AWS IoT Core), and Node-RED will be hosted on a reliable VPS (e.g., DigitalOcean or AWS EC2).
- **CI/CD Integration:** Leveraging Node-RED's built-in Git integration (Projects feature) to version-control the flow files (`flows.json`). This enables continuous

integration and automated deployment pipelines, ensuring that updates are safely pushed to production.

- **Monitoring & Alerting:** Integrating tools like Prometheus and Grafana to monitor server health, Node-RED CPU usage, and MQTT broker throughput continuously.

12.4 Security Enhancements

Securing the infrastructure against unauthorized access and cyber threats is a critical priority for physical traffic control:

- **Transport Layer Security:** Upgrading all communication protocols to enforce TLS encryption. ESP32 devices will communicate via MQTTS (Port 8883), and the Flutter API will strictly accept HTTPS connections to prevent packet sniffing.
- **Authentication & ACLs:** The MQTT broker will be secured using strict Access Control Lists (ACLs), ensuring that a compromised ESP32 at one intersection cannot publish commands to another. The Node-RED Dashboard will be protected via OAuth2 or JWT-based authentication.
- **Credential Management:** All sensitive tokens (such as the Telegram Bot API key) will be strictly managed using OS-level environment variables and Node-RED's encrypted credential store, removing all plaintext keys from the exportable flow JSON.
- **API Rate Limiting:** Implementing robust rate-limiting middlewares on the `/sanad` HTTP inbound endpoint to protect the system against DDoS attacks or accidental rapid-fire triggers from the mobile application.

13. References

[1] Node-RED, "Node-RED Documentation: Core Nodes and Flow Architecture," OpenJS Foundation. [Online]. Available: <https://nodered.org/docs/>

[2] OASIS, "MQTT Version 5.0 Specification," OASIS Standard. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

[3] FlowFuse, "Node-RED Dashboard 2.0 Documentation," Node-RED Community. [Online]. Available: <https://dashboard.flowfuse.com/>

[4] Telegram Core, "Telegram Bot API - Webhooks and sendMessage functionality," Telegram. [Online]. Available: <https://core.telegram.org/bots/api>

[5] Espressif Systems, "ESP32 Technical Reference Manual (IoT Microcontrollers)," Espressif. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>

[6] EMQ, "MQTTX: Elegant Cross-platform MQTT Desktop Client Documentation," MQTTX.app. [Online]. Available: <https://mqttx.app/docs/>

14. Appendices

[Instructions for student: Appendices hold material that is too large or detailed for the main text. Number figures and tables in the appendices separately (Figure A.1, Table B.1).]

Appendix A: Complete Node-RED Flow Export

```
[
  {
    "id": "sanad_main_flow_01",
    "type": "tab",
    "label": "Sanad - Adaptive Traffic Control",
    "disabled": false,
    "info": "Main event-driven flow managing ESP32 telemetry, HTTP Flutter API, and Telegram alerts."
  },
  {
    "id": "mqtt_in_telemetry",
    "type": "mqtt in",
    "z": "sanad_main_flow_01",
    "name": "ESP32 Telemetry In",
    "topic": "sanad/traffic/data",
    "qos": "0",
    "datatype": "json",
    "broker": "local_broker_config"
  }
]
```

Appendix B: Custom CSS / HTML Snippets (Dashboard Template Nodes)

```
<!-- Dark Glassmorphism UI snippet for Dashboard custom status cards -->
<style>
.sanad-glass-card {
  background: rgba(20, 20, 20, 0.75);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border-radius: 16px;
  border: 1px solid rgba(255, 255, 255, 0.1);
  padding: 24px;
  color: #ffffff;
  text-align: center;
  box-shadow: 0 8px 32px 0 rgba(0, 0, 0, 0.3);
}
.alert-text {
  color: #ff4757;
  font-weight: 700;
  font-size: 1.2em;
}
</style>
```

```
<div class="sanad-glass-card">
  <h2>Intersection Status</h2>
  <p>Active Emergency Override: <span class="alert-
text">{{msg.payload.state}}</span></p>
</div>
```